

1. DFS (Working)

```
edge(a,b).
edge(a,c).
edge(b,d).
edge(b,e).
edge(c,f).

dfs(Goal, Goal, Path, Path).

dfs(Current, Goal, Visited, Path) :-
    edge(Current, Next),
    \+ member(Next, Visited),
    dfs(Next, Goal, [Next|Visited], Path).
```

Query:

```
dfs(a,f,[a],P).
```

2. BFS (Working)

```
edge(a,b).
edge(a,c).
edge(b,d).
edge(b,e).
edge(c,f).

bfs([[Goal|Path]|_], Goal, [Goal|Path]).

bfs([[Current|Path]|Rest], Goal, Solution) :-
    findall([Next,Current|Path],
        (edge(Current, Next), \+ member(Next,[Current|Path])),
        NewPaths),
    append(Rest, NewPaths, Queue),
    bfs(Queue, Goal, Solution).
```

Query:

```
bfs([[a]], f, P).
```

3. Combined DFS + BFS

```
edge(a,b).
edge(a,c).
edge(b,d).
edge(b,e).
edge(c,f).

dfs(Goal, Goal, Path, Path).
dfs(Current, Goal, Visited, Path) :-
    edge(Current, Next),
    \+ member(Next, Visited),
    dfs(Next, Goal, [Next|Visited], Path).

bfs([[Goal|Path]|_], Goal, [Goal|Path]).
bfs([[Current|Path]|Rest], Goal, Solution) :-
    findall([Next,Current|Path],
        (edge(Current, Next), \+ member(Next,[Current|Path])),
        NewPaths),
    append(Rest, NewPaths, Queue),
    bfs(Queue, Goal, Solution).
```

Query:

```
dfs(a,f,[a],P).    bfs([[a]], f, P).
```

4. Best First Search

```
edge(a,b).
```

```

edge(a,c).
edge(b,d).
edge(b,e).
edge(c,f).

h(a,6).
h(b,4).
h(c,2).
h(d,0).
h(e,0).
h(f,0).

best([[Goal|Path]|_], Goal, [Goal|Path]).

best(Paths, Goal, Solution) :-
    select_best(Paths, BestPath, Rest),
    BestPath = [Current|Path],
    findall([Next,Current|Path],
        (edge(Current, Next), \+ member(Next, [Current|Path])),
        NewPaths),
    append(Rest, NewPaths, Updated),
    best(Updated, Goal, Solution).

select_best([Path], Path, []).

select_best([H|T], Best, [H|Rest]) :-
    select_best(T, Best, Rest),
    better(Best, H).

select_best([H|T], H, T) :-
    select_best(T, Best, _),
    better(H, Best).

better([A|_],[B|_]) :-
    h(A,HA), h(B,HB),
    HA =< HB.

```

Query:

```
best([[a]], f, P).
```

5.8 Queens

```

safe([]).
safe([Q|Qs]) :-
    safe(Qs),
    noattack(Q, Qs, 1).

noattack(_, [], _).
noattack(Q, [Q1|Qs], D) :-
    Q =\= Q1,
    abs(Q-Q1) =\= D,
    D1 is D + 1,
    noattack(Q, Qs, D1).

queens(N, Qs) :-
    numlist(1, N, List),
    permutation(List, Qs),
    safe(Qs).

```

Query:

```
queens(8, Q).
```